
Argentum

DOCUMENTACIÓN TÉCNICA

TALLER DE PROGRAMACIÓN

Integrantes

Thiago Reller	Pad. #111067
Federico Tomás Parsons	Pad. #111250
Agustin Atilio Ferronato	Pad. #111991
Sara Lucia Galeano	Pad. #112120

Índice

1. Introducción	3
2. Visión general de la arquitectura	3
3. Arquitectura del servidor	4
3.1. Diagrama de clases	4
3.2. Clases principales	4
3.2.1. Server	4
3.2.2. Acceptor (Thread)	5
3.2.3. ClientConnection (Thread)	5
3.2.4. Receiver (Thread)	5
3.2.5. Sender (Thread)	5
3.2.6. Game (Thread)	5
3.2.7. Command y CommandFactory	6
3.2.8. Character	6
3.2.9. NPC y NPCs concretos	6
3.2.10. Biome	6
3.2.11. City	6
3.2.12. PlayerRepository	6
3.2.13. SenderQueueMonitor	6
3.3. Flujo de una conexión	7
3.4. Modelo de hilos del servidor	7
4. Arquitectura del cliente	9
4.1. Diagrama de clases	9
4.2. Clases principales	9
4.2.1. Client	9
4.2.2. ClientSender (Thread)	9
4.2.3. ClientReceiver (Thread)	9
4.2.4. Gameloop	9
4.2.5. GameModel	10
4.3. Eventos manejados	10
4.3.1. GameController	11
4.3.2. GameWindow	11
4.3.3. Renderizado (Algoritmo del Pintor)	11
4.3.4. Audio	13
4.4. Lobby inicial (Qt)	13
5. Protocolo de comunicación	14
5.1. Diseño general	14
5.2. Formato de cada mensaje	14
5.3. Serialización de tipos	14
5.4. Tabla de comandos (Cliente → Servidor)	14
5.5. Tabla de eventos (Servidor → Cliente)	15
5.6. Ejemplo de flujo de mensajes	16
5.7. Parsers	18

6. Persistencia	19
6.1. Diseño	19
6.2. Formato del archivo de índice	19
6.3. Formato del archivo de datos	19
6.4. Flujo de guardado	20
7. Editor de mapas	21
7.1. Clases principales	21
7.2. Formato de mapa TOML	21
7.3. Cálculo de colisiones	22
8. Clases compartidas (common)	24
8.1. Player	24
8.2. Inventory	24
8.3. Formulas	24
8.4. Thread, Socket, Queue	24
8.5. DTOs y variant	25

1. Introducción

Este manual está dirigido a desarrolladores que quieran comprender la arquitectura de Argentum y eventualmente continuar con su desarrollo. Se describen las decisiones de diseño, la estructura del código, el modelo de hilos, el formato del protocolo de comunicación y el formato de los archivos de persistencia.

Los diagramas que se mencionan a lo largo del documento fueron generados con PlantUML

2. Visión general de la arquitectura

Argentum sigue una arquitectura **cliente-servidor** clásica con un protocolo binario sobre TCP. El servidor mantiene el estado del mundo, ejecuta la lógica del juego y persiste los datos de los jugadores. El cliente renderiza la interfaz, reproduce audio y envía los comandos del jugador al servidor.

El proyecto se divide en cuatro módulos:

- **common/** - Biblioteca compartida compilada como `taller_common`. Contiene el protocolo (DTOs, parsers, códigos de operación), las clases de datos del jugador (`Player`, `Inventory`, `Formulas`), las primitivas de concurrencia (`Thread`, `Socket`, `Queue`) y utilidades (`ConstantRateLoop`, `CollisionUtils`).
- **server/** - Biblioteca `taller_server_lib` más el ejecutable `taller_server`. Contiene la lógica del juego: `Game`, `Acceptor`, `ClientConnection`, `Character`, `NPC`, biomas, ciudades y el repositorio de persistencia.
- **client/** - Biblioteca `taller_client` más el ejecutable `taller_client`. Contiene el cliente gráfico: `GameLoop`, `GameModel`, `GameWindow`, `GameController`, el sistema de audio, el chat y las vistas HUD.
- **editor/** - Ejecutable `taller_editor`. Editor de mapas que utiliza Qt para la interfaz y SDL2 para el renderizado.
- **tests/** - Tests unitarios con GoogleTest sobre el protocolo y la persistencia.

3. Arquitectura del servidor

3.1. Diagrama de clases

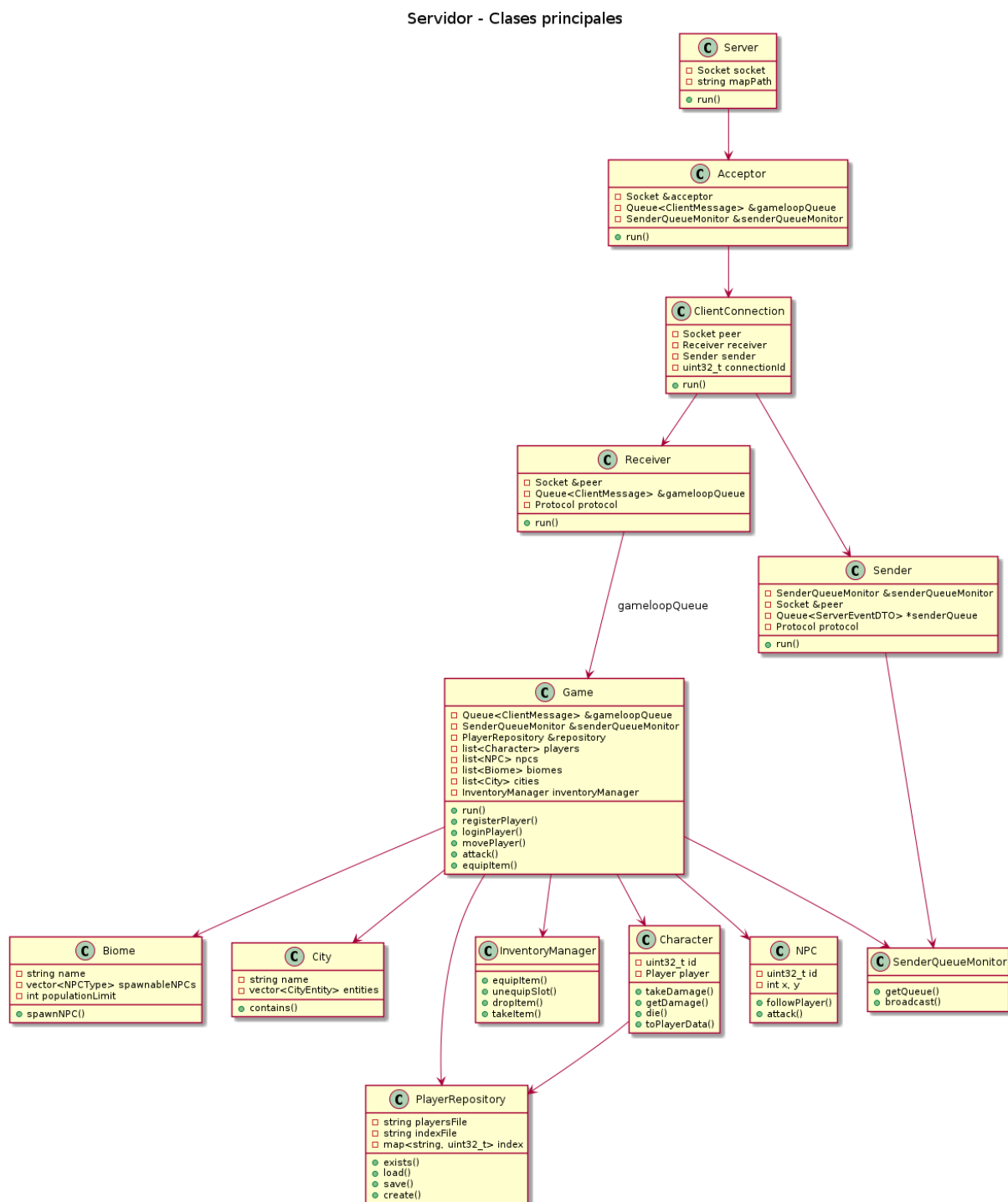


Figura 1: Diagrama de clases del servidor

3.2. Clases principales

3.2.1. Server

Punto de entrada. Recibe un puerto y la ruta al mapa. Crea el **Socket** del servidor, lo pone en escucha, y lanza el **Acceptor** y el **Game** como hilos separados. Espera a que ambos terminen

para finalizar.

3.2.2. Acceptor (Thread)

Hereda de **Thread**. En su método **run()** acepta conexiones entrantes en un bucle. Por cada nueva conexión crea un **ClientConnection** y lo inicia. También se encarga de recolectar (**reap**) las conexiones que ya terminaron, y de matar (**kill**) todas al recibir la señal de cierre.

3.2.3. ClientConnection (Thread)

Agrupar el **Receiver** y el **Sender** de un cliente. Es dueña del **Socket** de ese cliente. Al arrancar lanza ambos hilos. Al cerrar, los detiene y cierra el socket.

3.2.4. Receiver (Thread)

Lee del socket usando la clase **Protocol**. Cada mensaje entrante es un **ClientCommandDTO**. El **Receiver** lo envuelve en un **ClientMessage** (que agrega el **connectionId**) y lo encola en la **gameloopQueue**, que es compartida por todos los clientes. Si ocurre un error de comunicación, corta el bucle y muere.

3.2.5. Sender (Thread)

Obtiene una cola de eventos propia desde el **SenderQueueMonitor**. En su bucle desencola **ServerEventDTO** de esa cola y los envía por el socket usando **Protocol**. Esto asegura que cada cliente recibe solo los eventos que le corresponden y que un envío lento no bloquee a otros clientes.

3.2.6. Game (Thread)

Es el corazón del servidor y ejecuta el **game loop** principal. Su método **run()** ejecuta iteraciones de un **ConstantRateLoop**. En cada iteración:

1. **Procesa mensajes:** Desencola todos los **ClientMessage** de la **gameloopQueue** y llama a **execute()** que despacha según el tipo de comando. Cada comando tiene una clase asociada que implementa **Command**, creadas por **CommandFactory**.
2. **Mueve jugadores:** Llama a **movePlayers()** que actualiza las posiciones de todos los **Character** que están en movimiento, verificando colisiones.
3. **Mueve NPCs:** Llama a **makeNPCsFollowPlayers()** para que los NPCs persigan al jugador más cercano. También **appearNPCs()** para spawnear nuevos NPCs según el límite de población de cada bioma.
4. **Mueve entidades de ciudad:** Análogo a NPCs pero para sacerdotes, comerciantes y banqueros.
5. **Envía mensajes:** Vacía la lista **messagesToSend** distribuyendo cada evento al **SenderQueueMonitor**.
6. **Persiste:** Cada 300 ticks guarda el estado de todos los jugadores mediante **PlayerRepository**.

El **Game** mantiene las siguientes estructuras de datos principales:

- **players** - `unordered_map<uint32_t, unique_ptr<Character>` con todos los personajes conectados.

colisionables - Lista de punteros a **Colisionable** (incluye **Character** y ciertos NPCs) para verificar colisiones de forma uniforme.

biomes - Lista de biomas cargados del mapa. Cada bioma conoce qué NPCs spawnear y su límite de población.

cities - Lista de ciudades con sus zonas seguras y NPCs interactivos.

npcs - Lista de todos los NPCs vivos en el mundo.

connectionToPlayer y **playerToConnection** - Mapping bidireccional entre **connectionId** y **playerId**.

3.2.7. Command y CommandFactory

Cada tipo de comando del protocolo tiene una clase que implementa la interfaz **Command** con un método **execute(Game&)**. **CommandFactory** recibe un **ClientCommandDTO** (variant) y devuelve el **Command** concreto. Esto permite que **Game** no conozca los detalles de cada comando.

Comandos implementados: **AttackCommand**, **MovePlayerCommand**, **PlayerStoppedCommand**, **EquipCommand**, **UnequipCommand**, **DropItemCommand**, **TakeItemCommand**, **RegisterPlayerCommand**, **LoginPlayerCommand**, **ExitCommand**, **GlobalChatMessageCommand**.

3.2.8. Character

Representa a un jugador dentro del servidor. Hereda de **Colisionable** e implementa su interfaz (**colisionaCon()**, **getX()**, **getY()**, **getAncho()**, **getAlto()**). Delega los atributos de personaje a la clase **Player** (de common) y agrega métodos específicos del servidor como **takeDamage()**, **dropGoldOnDeath()**, **toPlayerData()** para persistencia, y métodos de conversión a DTOs (**toPlayerInfoEvent()**, **toPlayerAppeared()**).

3.2.9. NPC y NPCs concretos

NPC es la clase base para las criaturas. Tiene posición, tipo, stats y lógica de movimiento (persigue al jugador más cercano en un radio). Cada tipo concreto (**Goblin**, **Skeleton**, **Spider**, **Zombie**, **Orc**, **Golem**, **Giant**, **Elf**, **GreatReamer**) hereda de **NPC** y define sus propios stats y sprite. También existen NPCs de ciudad (**Banker**, **Priest**, **Trader**) que heredan de **CityEntity** y no atacan al jugador.

3.2.10. Biome

Carga sus parámetros desde el archivo TOML del mapa: nombre, textura de piso, lista de NPCs que pueden spawnear, límite de población y prioridad de renderizado.

3.2.11. City

Define una ciudad: nombre, posición, zona segura (rango de tiles donde no se puede atacar) y lista de **CityEntity**s que la habitan.

3.2.12. PlayerRepository

Implementa la persistencia de jugadores. Usa dos archivos binarios (índice y datos). Ver sección 6.

3.2.13. SenderQueueMonitor

Mantiene un mapa de **connectionId** → **Queue<ServerEventDTO>**. Cuando **Game** produce un evento, llama a **broadcast()** para enviarlo a todos, o a **sendToClient()** para enviarlo a uno en particular. Cada **Sender** obtiene su cola individual al iniciar.

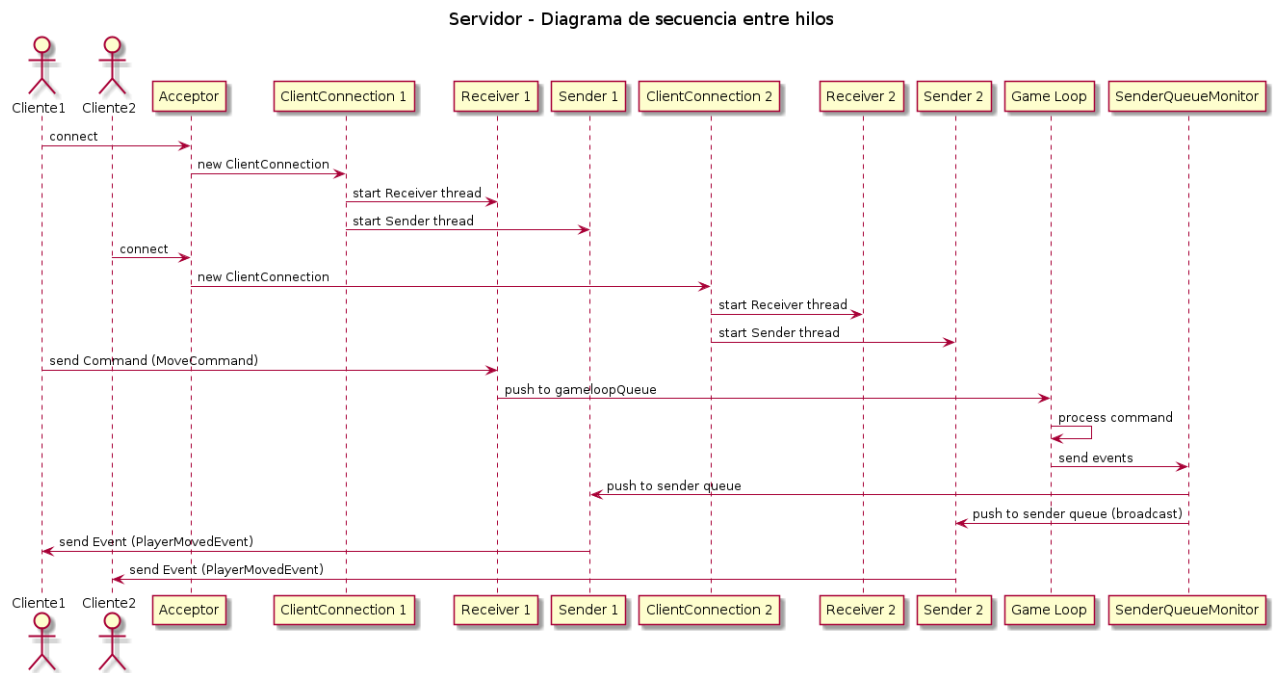


Figura 2: Diagrama de secuencia de hilos del servidor

3.3. Flujo de una conexión

1. El cliente se conecta al puerto del servidor.
2. **Acceptor** acepta la conexión, le asigna un `connectionId` y crea un **ClientConnection**.
3. **ClientConnection** crea **Receiver** y **Sender**, y les pasa el socket duplicado.
4. **Receiver** comienza a leer comandos. **Sender** obtiene su cola de eventos del monitor.
5. El cliente envía **RegisterPlayerCommand** o **LoginPlayerCommand**. **Game** lo procesa, crea un **Character** y registra el mapping.
6. A partir de ahí, el cliente envía comandos y recibe eventos hasta que envía **ExitCommand** o se desconecta.
7. **Game** detecta la desconexión, elimina al jugador y persiste sus datos.

3.4. Modelo de hilos del servidor

El servidor utiliza los siguientes hilos:

- **Hilo principal:** Crea el socket, lanza **Acceptor** y **Game**, y espera a que terminen con `join()`.
- **Acceptor:** Un hilo dedicado a aceptar conexiones. Por cada cliente crea un **ClientConnection** y lo lanza.
- **ClientConnection:** Un hilo por cliente conectado. Su único propósito es lanzar y coordinar **Receiver** y **Sender**. Cuando ambos terminan, **ClientConnection** termina también.
- **Receiver:** Un hilo por cliente. Lee del socket y escribe en la `gameLoopQueue`.
- **Sender:** Un hilo por cliente. Lee de su cola de eventos y escribe en el socket.

- **Game:** Un hilo que ejecuta el game loop principal a tasa constante.

La sincronización se logra mediante:

- `Queue<T>` bloqueante: las operaciones `push()` y `pop()` están sincronizadas internamente con mutex y condition variable.
- `SenderQueueMonitor` con mutex propio para acceder al mapa de colas de forma segura.
- El cierre coordinado usa `ShutdownEvent` y variables atómicas (`keepRunning`) que cada hilo verifica en su bucle.

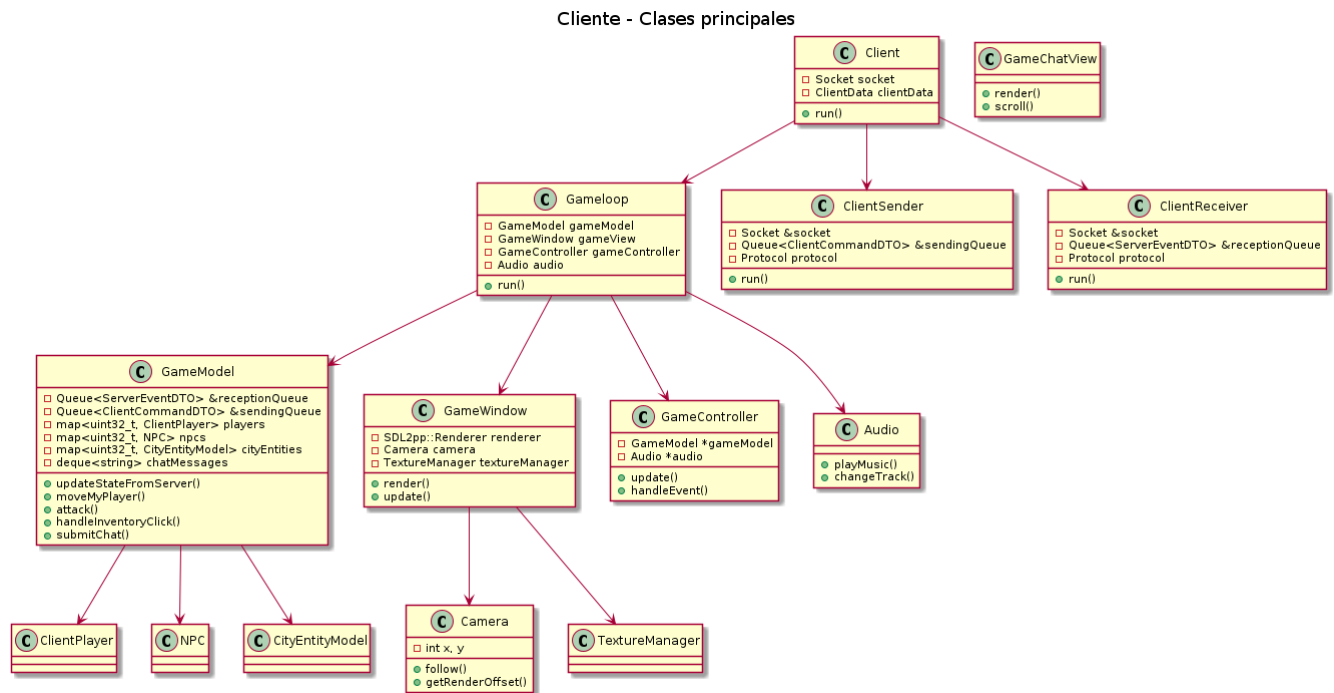


Figura 3: Diagrama de clases del cliente

4. Arquitectura del cliente

4.1. Diagrama de clases

4.2. Clases principales

4.2.1. Client

Punto de entrada del lado del cliente. Conecta el socket contra el servidor usando los datos provistos por el menú inicial (hostname, puerto, nombre de personaje, raza y clase). Luego lanza `ClientSender` y `ClientReceiver` en hilos separados, y ejecuta `Gameloop` en el hilo principal.

4.2.2. ClientSender (Thread)

Toma `ClientCommandDTO` de la `sendingQueue` y los envía por el socket usando `Protocol`. Si ocurre un error, notifica al `ShutdownEvent`.

4.2.3. ClientReceiver (Thread)

Lee `ServerEventDTO` del socket usando `Protocol` y los encola en la `receptionQueue`. Si ocurre un error, notifica al `ShutdownEvent`.

4.2.4. Gameloop

Ejecuta en el hilo principal y coordina tres componentes siguiendo una variante de MVC:

- **GameModel:** El modelo. Actualiza el estado del juego a partir de los eventos del servidor. También recibe acciones del controlador y produce comandos para enviar al servidor.
- **GameWindow:** La vista. Renderiza el juego con SDL2: mapa, jugadores, NPCs, efectos, HUD, chat e inventario.

- **GameController:** El controlador. Procesa eventos SDL (teclado, mouse) y llama a **GameModel** para traducirlos a comandos.

El gameloop corre a una tasa fija usando **ConstantRateLoop**. En cada iteración:

1. **GameModel.updateStateFromServer()** - Vacía la **receptionQueue** y procesa cada evento con su handler específico (sobrecarga del método **handle()** para cada tipo de DTO).
2. **GameController.update()** - Procesa eventos SDL (input del usuario).
3. **GameWindow.update()** - Renderiza el frame actual.

4.2.5. GameModel

Contiene el estado local del juego:

- **players** - Mapa de `uint32_t` → **ClientPlayer** con todos los jugadores visibles.
- **npcs** - Mapa de NPCs visibles.
- **cityEntities** - Entidades de ciudad (sacerdotes, comerciantes, banqueros).
- **groundItemManager** - Ítems en el suelo.
- **chatMessages** - Historial de mensajes del chat.
- **myPlayerID** - ID del jugador local.

El procesamiento de eventos se hace mediante **visitor por sobrecarga**: hay un método **handle()** para cada tipo concreto de **ServerEventDTO**. El método **updateStateFromServer()** desempaqueta el variant y llama al handler correspondiente usando `std::visit`.

4.3. Eventos manejados

- | | |
|------------------------------------|-------------------------------------|
| ▪ PlayerMovedEventDTO | ▪ PrivateMessageEventDTO |
| ▪ PlayerAppearedEventDTO | ▪ TextureInfoEventDTO |
| ▪ PlayerStoppedEventDTO | ▪ RegisterPlayerEventDTO |
| ▪ PlayerRemovedEventDTO | ▪ PlayerListEventDTO |
| ▪ PlayerInfoEventDTO | ▪ GroundItemAppearedEventDTO |
| ▪ InventoryUpdateEventDTO | ▪ GroundItemRemovedEventDTO |
| ▪ NPCAppearedEventDTO | ▪ GroundItemsListEventDTO |
| ▪ NPCMovedEventDTO | ▪ CityEntityAppearedEventDTO |
| ▪ NPCStoppedEventDTO | ▪ CityEntityMovedEventDTO |
| ▪ NpcDefeatedEventDTO | ▪ CityEntityStoppedEventDTO |
| ▪ ChatMessageEventDTO | ▪ PlayerDieEventDTO |
| ▪ GlobalChatMessageEventDTO | |

4.3.1. GameController

Procesa eventos SDL. En cada frame:

- Verifica teclas presionadas (WASD/ flechas para movimiento) y genera comandos `MoveCommand` / `PlayerStopCommand`.
- Detecta clicks del mouse: izquierdo para atacar a un NPC o jugador en las coordenadas clickeadas, derecho para interactuar con el inventario.
- Detecta la tecla Enter para abrir/cerrar el chat y la tecla M para cambiar la pista musical.

4.3.2. GameWindow

Clase que centraliza el renderizado con SDL2. Componentes:

- **Camera:** Sigue al jugador local, manteniéndolo centrado en la pantalla. Calcula el offset de renderizado para cada entidad.
- **TextureManager:** Carga y gestiona todas las texturas desde archivos de configuración TOML. Soporta layouts de sprites (frames de animación).
- **TextureMapper:** Mapea tipos de entidad a texturas concretas según la configuración.
- **InventoryPanel:** Renderiza el panel de inventario con los ítems equipados y los del inventario.
- **Renderizado por capas:** Las texturas se renderizan en orden de prioridad (piso, objetos, entidades, efectos) según la configuración TOML.

4.3.3. Renderizado (Algoritmo del Pintor)

Para renderizar los objetos en pantalla correctamente se implementó el algoritmo del pintor (Painter's algorithm). Este algoritmo establece un orden de dibujo de las texturas en función de su distancia relativa al observador, permitiendo que los objetos más cercanos se superpongan visualmente a los más lejanos.

Dado que Argentum utiliza una perspectiva isométrica, fue necesario incluir un sistema de prioridades para garantizar una representación adecuada de la profundidad de la escena. Para lograr dicho objetivo, las texturas de menor prioridad se renderizan primero. Dentro de cada capa, se ejecuta el algoritmo del pintor y se ordena cada estructura de menor a mayor de acuerdo a la última columna y fila que ocupan en el mapa.

Las entidades (NPCs y jugadores) se renderizan utilizando la misma técnica. Sin embargo, poseen una prioridad superior con respecto a las texturas de terreno (prioridad 0), lo que garantiza que las mismas aparezcan correctamente por encima del suelo y de los elementos que corresponda según su posición en la escena.

Las estructuras a renderizar se almacenan en un contenedor de la forma `std::vector<std::map<std::pair<int, int>, std::vector<GridItem> tilesToRender`. Cada posición en el vector representa una prioridad. En esta estructura, la clave es una tupla que representa la última celda (fila y columna) del mapa ocupada por la textura.

Cada textura se representa en memoria mediante un vector de objetos `GridItem`. Dado que el mapa se encuentra dividido en celdas de 32x32 píxeles, resulta necesario particionar las texturas para que puedan distribuirse correctamente sobre dichas celdas. Para ello, cada textura se subdivide en múltiples fragmentos, donde cada fragmento corresponde a una celda en el mapa.

Cada `GridItem` almacena la información necesaria para renderizar uno de estos fragmentos, incluyendo los desplazamientos horizontales y verticales respecto de la imagen original. Gracias a esta representación, es posible reconstruir visualmente texturas de gran tamaño a partir de

múltiples celdas del mapa, manteniendo tanto su posición relativa como su integración con el sistema de prioridades utilizado durante el renderizado.

El código que permite el renderizado de las estructuras y entidades es el siguiente:

```
void GameWindow::renderWorld(unsigned int it) {
    renderCommonGround();
    renderGroundItems();

    std::vector<RenderableEntity *> sortedEntities;
    getSortedEntities(sortedEntities);

    for (size_t i = 0; i < tilesToRender.size(); i++) {
        auto &priority = tilesToRender[i];

        for (auto &[pair, items] : priority) {
            int maxRow = pair.first;
            int yMax = (maxRow - maxSize / 2 + 1) * gridSize;

            for (auto *entity : sortedEntities) {
                if (!entity->rendered() &&
                    entity->get_y() + entity->get_h() < yMax &&
                    entity->hasPriority(i)) {
                    entity->render(*renderer, camera, it);
                }
            }

            for (auto &item : items) {
                SDL2pp::Rect dstRect = camera.toScreen(
                    (item.i - maxSize / 2) * gridSize,
                    (item.j - maxSize / 2) * gridSize, gridSize, gridSize);

                SDL2pp::Rect srcRect = {item.x_start, item.y_start,
                                         item.x_end - item.x_start,
                                         item.y_end - item.y_start};
                renderer->Copy(textureMapper->getTexture(item.texture_id), srcRect,
                               dstRect);
            }
        }

        for (auto *entity : sortedEntities) {
            if (!entity->rendered() && entity->hasPriority(i)) {
                entity->render(*renderer, camera, it);
            }
        }
    }

    for (auto & entity : sortedEntities) {
        if (!entity->rendered()) {
            entity->render(*renderer, camera, it);
        }
    }
}
```

Listing 1: Función renderWorld en GameWindow

4.3.4. Audio

Reproduce música de fondo usando SDL2_mixer. Soporta 3 pistas diferentes. La tecla M cambia entre ellas. No hay efectos de sonido (SFX) en la versión actual.

4.4. Lobby inicial (Qt)

Antes del gameloop SDL, el cliente muestra un menú desarrollado con Qt que permite:

- **Login:** Ingresar nombre de usuario existente.
- **Registro:** Crear personaje nuevo con nombre, raza y clase.
- **Fondo animado:** La ventana tiene un fondo con animación mientras se espera la conexión.

Los datos ingresados se pasan al **Client** a través de **ClientData**. Si el login es exitoso, la ventana Qt se cierra y comienza el gameloop SDL.

5. Protocolo de comunicación

5.1. Diseño general

El protocolo es **binario** y corre sobre TCP. Los mensajes se dividen en dos categorías:

- **Comandos** (cliente → servidor): Enviados por el `ClientSender`, recibidos por el `Receiver` del servidor.
- **Eventos** (servidor → cliente): Enviados por el `Sender`, recibidos por el `ClientReceiver`.

Los DTOs y parsers se encuentran en `common/`. Cada DTO tiene un `getCode()` que devuelve su opcode. Los mensajes se serializan mediante `NetworkUtils` que escribe los campos en big-endian uno tras otro.

5.2. Formato de cada mensaje

+-----+	+-----+	+-----+
Opcode (1 byte)	Largo datos (2B)	Datos (variable)
+-----+	+-----+	+-----+

Listing 2: Formato general

- **Opcode:** 1 byte. Identifica el tipo de mensaje. Los comandos van de 0x10 a 0x1C, los eventos de 0x80 a 0x97.
- **Largo:** uint16 en big-endian. Indica cuántos bytes de datos siguen (sin incluir opcode ni el propio largo).
- **Datos:** Los campos del DTO serializados en orden. Los strings se serializan como: `uint16 largo + chars`. Los enteros se serializan en big-endian (network byte order).

5.3. Serialización de tipos

- **uint8:** 1 byte.
- **uint16:** 2 bytes, big-endian.
- **int16:** 2 bytes, big-endian, complemento a 2.
- **uint32:** 4 bytes, big-endian.
- **string:** uint16 con el largo, seguido de los bytes del texto (sin null terminator).
- **Direction:** uint8 (0=Up, 1=Down, 2=Left, 3=Right).

5.4. Tabla de comandos (Cliente → Servidor)

A continuación se detallan los paquetes binarios de comandos enviados por el cliente y las estructuras de sus campos asociados:

Código	Comando	Campos y Tipos Estructurados
0x10	RegisterPlayerCommand	nombre (string), raza (string), clase (string)

Código	Comando	Campos y Tipos Estructurados
0x11	LoginPlayerCommand	nombre (string)
0x12	MeditateCommand	—
0x13	PrivateMessageCommand	destino (string), mensaje (string)
0x14	MoveCommand	playerId (u32), dirección (u8)
0x15	ExitCommand	—
0x16	PlayerStopCommand	playerId (u32)
0x17	EquipCommand	playerId (u32), slot (u8)
0x18	UnequipCommand	playerId (u32), equipSlot (u8)
0x19	DropItemCommand	playerId (u32), slot (u8)
0x1A	TakeItemCommand	playerId (u32)
0x1B	GlobalChatMessageCommand	playerId (u32), mensaje (string)
0x1C	AttackCommand	playerId (u32), targetX (i16), targetY (i16)

5.5. Tabla de eventos (Servidor → Cliente)

Detalle de las tramas de eventos serializadas por el servidor para la actualización del estado del juego en el frontend:

Código	Evento	Campos y Tipos Estructurados
0x80	ChatMessageEvent	tipo (string), mensaje (string)
0x81	NPCDefeatedEvent	npcId (u32)
0x82	NPCRespawnedEvent	—
0x83	PlayerMovedEvent	playerId (u32), x (i16), y (i16), dir (u8)
0x84	RegisterPlayerEvent	playerId (u32)
0x85	PlayerListEvent	cantidad (u8), por jugador: id (u32), nombre (string), raza (string), x (i16), y (i16), dir (u8)
0x86	PlayerInfoEvent	playerId (u32), hp (u32), maxHp (u32), mana (u32), maxMana (u32), oro (u32), nivel (u32), exp (u32)
0x87	PlayerAppearedEvent	playerId (u32), nombre (string), raza (string), x (i16), y (i16), dir (u8)
0x88	PlayerStoppedEvent	playerId (u32), x (i16), y (i16)
0x89	PlayerRemovedEvent	playerId (u32)
0x8A	NPCAppearedEvent	npcId (u32), tipo (u8), x (i16), y (i16)
0x8B	InventoryUpdateEvent	items[20] (20 u8), weapon (u8), armor (u8), helmet (u8), shield (u8)
0x8C	TextureInfoEvent	—
0x8D	GroundItemAppearedEvent	itemId (u8), x (i16), y (i16)
0x8E	GroundItemRemovedEvent	itemId (u8), x (i16), y (i16)
0x8F	GroundItemsListEvent	cantidad (u8), por ítem: itemId (u8), x (i16), y (i16)
0x90	PrivateMessageEvent	remitente (string), mensaje (string)
0x91	GlobalChatMessageEvent	playerId (u32), nombre (string), mensaje (string)
0x92	NPCMovedEvent	npcId (u32), x (i16), y (i16)
0x93	NPCStoppedEvent	npcId (u32), x (i16), y (i16)

Código	Evento	Campos y Tipos Estructurados
0x94	CityEntityAppearedEvent	entityId (u32), tipo (u8), x (i16), y (i16)
0x95	CityEntityMovedEvent	entityId (u32), x (i16), y (i16)
0x96	CityEntityStoppedEvent	entityId (u32), x (i16), y (i16)
0x97	PlayerDieEvent	playerId (u32)

5.6. Ejemplo de flujo de mensajes

A continuación se muestra la secuencia típica de mensajes para varias acciones del juego

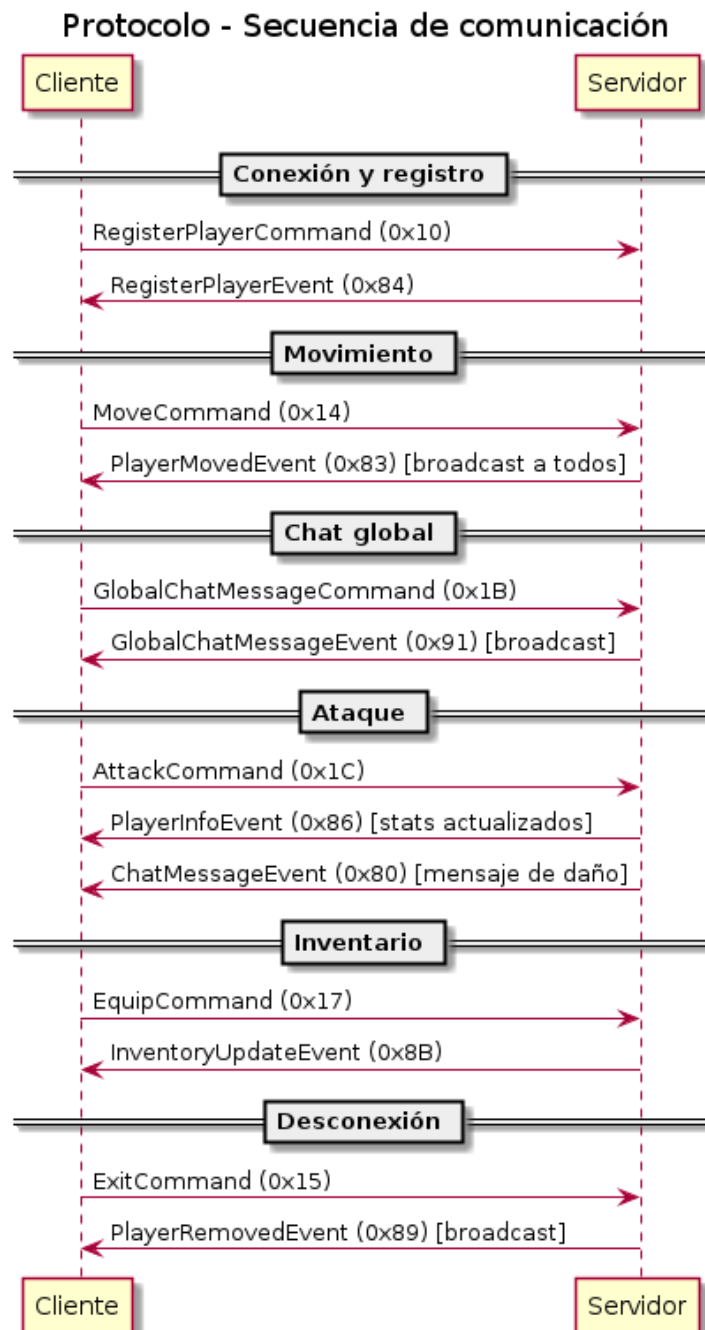


Figura 4: Secuencia de Mensajes

5.7. Parsers

El protocolo sigue un esquema de **Abstract Factory + Visitor**. Existe un parser por cada tipo de mensaje:

- **ClientCommandParser** - clase base para parsers de comandos.
- **ServerEventParser** - clase base para parsers de eventos.
- **RegisterAllParsers** - registro de todos los parsers en el **Protocol**.

Cada parser concreto (ej. **MoveCommandParser**) sabe leer los bytes del socket y construir el DTO correspondiente. El **Protocol** usa el opcode para seleccionar el parser adecuado de un mapa interno.

6. Persistencia

6.1. Diseño

La persistencia de jugadores se implementa en `PlayerRepository`. Se utilizan dos archivos binarios:

- `index.dat`: Mapa de nombre de jugador a offset dentro del archivo de datos. Se permite tenerlo entero en memoria por ser pequeño.
- `players.dat`: Datos serializados de cada jugador, escritos de forma contigua.

6.2. Formato del archivo de índice

```
+-----+-----+
| Campo | Tipo |
+-----+-----+
| Cantidad de entradas | uint32 (4 bytes) |
| Entrada 1: nombre (string), offset (uint32) |
| Entrada 2: nombre (string), offset (uint32) |
| ... | |
+-----+-----+
```

Listing 3: Estructura de `index.dat`

Donde `string` se serializa como: `uint16` con el largo + los bytes del nombre.

6.3. Formato del archivo de datos

Cada `PlayerData` se serializa en el siguiente orden:

```
nombre (string): uint16 largo + chars
raza (string): uint16 largo + chars
clase (string): uint16 largo + chars
x (int32): 4 bytes
y (int32): 4 bytes
direction (uint8): 1 byte
level (uint32): 4 bytes
hp (uint32): 4 bytes
maxHp (uint32): 4 bytes
mana (uint32): 4 bytes
maxMana (uint32): 4 bytes
experience (uint32): 4 bytes
gold (uint32): 4 bytes
strength (uint32): 4 bytes
agility (uint32): 4 bytes
constitution (uint32): 4 bytes
intelligence (uint32): 4 bytes
inventory (20 x u8): 20 bytes (un itemId por slot, 0 = vacio)
equippedWeapon (u8): 1 byte
equippedArmor (u8): 1 byte
equippedHelmet (u8): 1 byte
equippedShield (u8): 1 byte
-----
TOTAL: aproximadamente 100 bytes por jugador
```

Listing 4: Estructura de cada `PlayerData`

6.4. Flujo de guardado

1. Cada 300 ticks del game loop, **Game** itera sobre todos los **Character** y llama a **PlayerRepository.save()**.
2. **save()** busca el offset del jugador en el índice, se posiciona en **players.dat** en ese offset y sobrescribe los datos. Si es un jugador nuevo (no está en el índice), escribe al final del archivo y actualiza el índice.
3. Al desconectarse un jugador, también se guarda.
4. El índice se guarda en el destructor de **PlayerRepository**.

No se puede cargar el archivo completo en memoria (se asumen miles de jugadores), por lo que cada **load()** lee solo el registro correspondiente accediendo por offset.

7. Editor de mapas

El editor combina **Qt** (interfaz de usuario: menús, botones, barras) con **SDL2** (renderizado del mapa en una ventana). Está pensado para que el diseñador pueda crear mapas visualmente y exportarlos a formato TOML.

7.1. Clases principales

- **Editor:** Ventana principal Qt. Contiene la barra de herramientas, la grilla SDL y los paneles de configuración. Coordina la carga, edición y guardado de mapas.
- **Grid:** Modelo de la grilla. Almacena las celdas con sus texturas, biomas y propiedades (colisionable, spawn, ciudad).
- **GridSDL:** Renderiza la grilla usando SDL2. Maneja el hover, el zoom y el desplazamiento.
- **Tile:** Representa una celda individual: textura, bioma, altura, prioridad.
- **Biome:** Define un bioma con su nombre, textura de piso y parámetros asociados.
- **TextureMap:** Carga las texturas desde archivos de imagen y las mapea a IDs según la configuración TOML.
- **MapLoader:** Lee y escribe mapas en formato TOML. El mismo formato es usado por el servidor para cargar el mapa al iniciar.

7.2. Formato de mapa TOML

Los mapas se guardan en archivos TOML con la siguiente estructura:

```
[map]
COMMON_GROUND_TEXTURE_ID = 0
GRID_SIZE = 32
MAX_SIZE = 100

[[biomes]]
i_end = 52
i_init = 40
j_end = 53
j_init = 40
type = 'Forest'

[[biomes]]
i_end = 61
i_init = 55
j_end = 52
j_init = 40
type = 'Desert'

[[cities]]
i_end = 59
i_init = 42
j_end = 61
j_init = 56
type = 'City'

[[collidable_cells]]
i = 45
```

```
j = 48

[[collidable_cells]]
i = 45
j = 49

[[collidable_cells]]
i = 46
j = 48

[[collidable_cells]]
i = 46
j = 49

[[textures]]
i = 42
j = 41
priority = 1
txt_id = 2
```

Listing 5: Ejemplo de mapa.toml

7.3. Cálculo de colisiones

Muchas de las estructuras presentes en el mapa, tales como árboles, casas o rocas, no deben poder ser atravesadas por las entidades del juego. Una primera aproximación para resolver este problema consiste en asociar a cada textura una bandera (flag) que indique si la misma es colisionable o no. Sin embargo, esta solución resulta insuficiente debido a la perspectiva isométrica utilizada por Argentum.

En una escena isométrica, no necesariamente toda la superficie visible de una estructura debe participar en la detección de colisiones. Consideremos, por ejemplo, el caso de un árbol: resulta deseable que únicamente el tronco sea colisionable, mientras que la copa pueda ser atravesada visualmente por las entidades. Por lo tanto, marcar la textura completa como colisionable produciría resultados incorrectos.

Una segunda alternativa consiste en indicar qué porcentaje de la textura corresponde a una región colisionable. No obstante, esta solución presenta otro inconveniente: las texturas son imágenes que suelen contener grandes áreas transparentes. Si se define, por ejemplo, que el 25 % inferior de la textura de un árbol es colisionable, también se incluirían celdas completamente transparentes que no representan información visual relevante.

Para evitar este problema, se implementó un mecanismo basado en el canal alfa de la imagen. Durante la partición de la textura en celdas de 32×32 píxeles, se analiza individualmente cada una de ellas. Una celda se considera colisionable únicamente si la cantidad de información visual que contiene supera un determinado umbral.

Concretamente, para cada celda se calcula el promedio de los valores del canal alfa de todos sus píxeles. Dicho valor se normaliza en el intervalo $[0, 1]$, donde 0 representa una región completamente transparente y 1 una región completamente opaca. Si el promedio obtenido supera el umbral establecido, la celda se marca como colisionable; en caso contrario, se descarta.

El umbral utilizado en la implementación es 0.15. De esta manera, se eliminan automáticamente las regiones transparentes que no aportan información visual significativa, obteniendo una representación de colisiones mucho más precisa y acorde a la geometría percibida por el jugador.

La función encargada de calcular el peso promedio del canal alfa se ejecuta dentro del editor de mapas y se muestra a continuación:

```
float Grid::getAlphaChannelWeight(SDL2pp::Surface &surface,
                                SDL2pp::Rect region) {

    SDL2pp::Surface converted = surface.Convert(SDL_PIXELFORMAT_RGBA8888);

    SDL2pp::Surface::LockHandle lock = converted.Lock();

    long long totalPixels = region.w * region.h;
    long long alpha = 0;

    int pitch = converted.Get()->pitch / sizeof(UInt32);
    UInt32 *pixels = static_cast<UInt32 *>(converted.Get()->pixels);

    for (int y = region.y; y < region.y + region.h; y++) {
        for (int x = region.x; x < region.x + region.w; x++) {

            UInt32 pixel = pixels[y * pitch + x];

            UInt8 r, g, b, a;
            SDL_GetRGBA(pixel, converted.Get()->format, &r, &g, &b, &a);

            alpha += a;
        }
    }

    return static_cast<float>(alpha) / (totalPixels * 255.0f);
}
```

Listing 6: Función getAlphaChannelWeight en Grid

8. Clases compartidas (common)

8.1. Player

Clase que representa los datos de un jugador. Contiene: nombre, raza, clase, dirección, posición, stats (**PlayerStats**), inventario (**Inventory**). Es usada tanto en servidor (**Character** la envuelve) como en cliente (**ClientPlayer** la envuelve). Métodos importantes:

- **updateStats()** - Actualiza hp, mana, oro, nivel y experiencia desde un evento del servidor.
- **takeDamage()** - Resta vida y devuelve el daño real aplicado.
- **heal()** - Stub (devuelve 0, no implementado).
- **useMana()** - Stub (devuelve false).
- **gainExperience()** - Suma XP y sube de nivel si corresponde.
- **die()** - Devuelve los ítems del inventario para que caigan al suelo y marca al jugador como muerto.

8.2. Inventory

Arreglo de 20 slots (**array<uint8_t, MAX_INVENTORY_SLOTS>**). Cada slot almacena un **itemId** (0 = vacío). Provee métodos para agregar, remover, equipar y desequipar ítems. Controla que no se puedan equipar arma y vâro/bâculo simultâneamente.

8.3. Formulas

Clase con métodos estâticos que implementan todas las ecuaciones del juego segùn el enunciado. Todos los valores numéricos provienen de archivos de configuración TOML. Métodos principales:

- **calcularVidaMax()**, **calcularManaMax()**
- **calcularRecuperacionVida()**, **calcularRecuperacionMana()**
- **calcularRecuperacionMeditacion()**
- **calcularOroMax()**, **calcularOroExceso()**
- **calcularExperiencia()**, **calcularExperienciaMuerte()**
- **calcularDaño()**, **calcularEsquivo()**, **calcularCritico()**, **calcularDefensa()**
- **getRaceStats()**, **getPlayerClassStats()**

8.4. Thread, Socket, Queue

Primitivas de concurrencia y comunicaciôn:

- **Thread**: Clase base abstracta. Usa **std::thread**. Métodos: **start()**, **join()**. Las subclases implementan **run()**.
- **Socket**: RAII sobre un file descriptor de socket. Soporta **connect()**, **accept()**, **send()**, **recv()**, **shutdown()**. Lanza **LibError** en caso de error.
- **Queue<T>**: Cola bloqueante thread-safe. **push()** notifica, **pop()** bloquea hasta tener datos. Soporta cierre con **close()**.

8.5. DTOs y variant

Cada comando y evento es un `struct` con campos públicos y un método `getCode()`. Se agrupan en variants usando `std::variant`:

- `ClientCommandDTO` = variant de 13 tipos de comando.
- `ServerEventDTO` = variant de 23 tipos de evento.

Esto permite escribir código de despacho genérico con `std::visit` sin usar herencia polimórfica.